UNIVERSITÉ DE MONS

FACULTÉ DES SCIENCES

Refactoring GitHub Actions for Reuse

Abdellah BOUMAIZA

Master en Sciences Informatiques

abdellah.boumaiza@student.umons.ac.be

Academic year 2026–2027

Master's thesis carried out in the **Software Engineering Lab**, as part of the **Master en Sciences Informatiques**.

Supervisor: Prof. Tom MENS
Co-supervisor: Aref TALEBZADEH BARDSIRI

Faculté des Sciences

UMONS Université de Mons

The author, BOUMAIZA Abdellah, declares having complied with the ethical rules in force, including the University's charter on the use of Artificial Intelligence.

Contents

Introduction **4**

1 Reuse mechanisms in GitHub Actions **5**

2 Related work and its relevance **6**
2.1 Code clones in workflows 6
2.2 Practitioners' reuse practices 6
2.3 Refactoring and workflow smells 6

3 Existing tools **7**
3.1 YAML processing 7
3.2 Querying and transforming YAML 7
3.3 Workflow analysis tools 7

4 Research direction **8**
4.1 Candidate refactorings 8
4.2 Technical approach 8
4.3 Behaviour preservation 8

Bibliography **10**

Introduction

This thesis aims to design and implement an automated refactoring tool for GitHub Actions workflows that introduces the platform's reuse mechanisms, namely composite actions and reusable workflows, while preserving workflow behaviour. The scope is initially restricted to reuse-oriented refactorings.

Chapter 1

Reuse mechanisms in GitHub Actions

Before designing refactorings, it is necessary to establish which reuse mechanisms the platform already offers, since each of them defines a possible target for an automated transformation.

- **Actions** encapsulate a single reusable unit of logic, referenced in a step through `uses: owner/repo@ref`. They are widely adopted, mostly through the GitHub Marketplace.
- **Composite actions** group a sequence of steps in an `action.yml` file and execute within the calling job. They accept inputs and expose outputs, but cannot access the `secrets` context and require an explicit `shell` for every `run` step. They can be stored locally in the repository or in a separate one.
- **Reusable workflows**, declared with the `workflow_call` trigger, are invoked at the job level and therefore encapsulate entire jobs. They accept inputs, secrets and outputs, but secrets must be passed explicitly or through `secrets: inherit`.
- **Matrix strategies** execute a single job definition over several parameter values, which makes them suitable for replacing near-identical jobs within the same workflow.

These mechanisms operate at different levels of granularity, namely the step, the job and the workflow. This granularity determines which kind of duplication each of them can remove, and consequently which refactoring should be applied to a given clone.

Chapter 2

Related work and its relevance

2.1 Code clones in workflows

Cardoen, Mens and Decan [1] analysed 118K workflows and found clones in 67% of them, most of which are short sequences of two or three steps. They estimate that local composite actions could remove 83.7% of clones at file scope and 75.6% at repository scope, whereas reusable workflows would remove only about 2%, since they require entire jobs. *Relevance:* this study identifies *Extract Composite Action* as the refactoring with the highest potential and provides both refactoring candidates and a quantitative baseline. Its estimate, however, does not verify whether the reuse would preserve behaviour, which is precisely the gap this thesis addresses.

2.2 Practitioners' reuse practices

Onsori Delicheh et al. [4] surveyed 419 practitioners. Copy-paste remains common for reasons of convenience and control; 26.9% of respondents were unaware of composite actions and 25.4% unfamiliar with them, and reuse is often perceived as adding complexity. *Relevance:* the main barriers are awareness and effort rather than trust, which a tool generating reuse automatically can directly lower, provided that its output remains readable and that it suggests changes rather than imposing them.

2.3 Refactoring and workflow smells

Fowler [2] defines refactoring as a behaviour-preserving improvement of internal structure and catalogues refactorings together with the smells they remove; *Extract Function*, for instance, directly corresponds to *Extract Composite Action*. Khatami et al. [3] propose a catalogue of smells specific to GitHub Actions workflows. *Relevance:* these works provide the conceptual framework of the thesis, namely the definition of a refactoring, the link between smells and refactorings, and the duplication-related smells that the tool should target.

Chapter 3

Existing tools

A workflow refactoring tool requires three capabilities: the faithful parsing and serialisation of YAML, the querying and transformation of its structure, and an understanding of workflow semantics.

3.1 YAML processing

PyYAML is the reference YAML library for Python. It maps documents onto native dictionaries and lists, which facilitates analysis, but discards comments, key order and formatting upon serialisation. Moreover, it implements YAML 1.1, under which the `on` key of a workflow is interpreted as the boolean `True`. **ruamel.yaml**, in contrast, provides a round-trip mode in which documents are loaded into dedicated structures that retain comments, key order, quoting style and anchors. Its output thus remains close to the original, which is essential for developer acceptance and makes it the natural foundation for the tool.

3.2 Querying and transforming YAML

Whereas XML benefits from XPath and XQuery for querying and XSLT for transformation, YAML lacks standard equivalents. **JSONPath** and **JMESPath** support declarative queries over YAML documents once loaded as data, but provide no transformation capabilities. **yq** supports both querying and editing from the command line, yet offers limited control over formatting and has no notion of workflow semantics. A promising direction is therefore a lightweight query and rewriting layer, built in Python on ruamel.yaml and tailored to GitHub Actions workflows.

3.3 Workflow analysis tools

gawd [5], developed within the Software Engineering Lab, represents workflows as trees and identifies the items added, removed, moved, renamed or modified between two versions. It may inform the tool's internal representation and help verify that a refactoring yields exactly the intended changes. **actionlint** is a static checker that verifies workflow syntax, the types of `${{ }}` expressions, job dependencies, action inputs and embedded shell scripts. As its checks target errors rather than design issues such as duplication, it is better suited to validating refactored workflows. Finally, the **clone detector of Cardoen et al.** [1] identifies identical and near-identical fragments by hashing normalised workflow trees, thereby providing both a source of refactoring candidates and a baseline for evaluation.

Chapter 4

Research direction

4.1 Candidate refactorings

Following Fowler [2], a refactoring is defined as a transformation that improves the internal structure of a program without altering its observable behaviour. For GitHub Actions, this suggests several reuse-oriented refactorings: the extraction of duplicated steps into a local composite action (and its inverse, inlining), the extraction of duplicated jobs into a reusable workflow invoked through `workflow_call`, the splitting and merging of jobs and steps, and the decomposition of a complex workflow into two interdependent workflows. Each refactoring will be characterised by the duplication or smell it addresses, its preconditions, and the transformation it performs. Given that most clones consist of short step sequences, *Extract Composite Action* is the natural starting point.

4.2 Technical approach

Following the classical architecture of refactoring tools, each workflow is parsed with ruamel.yaml into a tree preserving its concrete syntax. This tree is then abstracted into a workflow model capturing jobs, steps, the dependency graph induced by `needs`, and the references contained in expressions. Refactoring opportunities are identified through queries on this model, the corresponding transformations are applied, and the result is serialised back to YAML together with any newly created file, such as an `action.yml`. The envisaged outcome is a Python command-line tool, comparable in design to gawd, capable of suggesting or applying refactorings.

4.3 Behaviour preservation

The principal challenge lies in guaranteeing that a refactored workflow remains behaviourally equivalent to the original. In GitHub Actions, this is non-trivial: each job executes on a separate runner and therefore does not share its file system; reusable workflows do not inherit secrets unless these are passed explicitly; and composite actions require a `shell` attribute on every `run` step and cannot access the `secrets` context directly. Such constraints will be formalised as preconditions to be verified before any refactoring is applied, and refactored workflows will additionally be validated with actionlint.

For instance, consider a duplicated step that reads a secret: `run: ./deploy.sh` with `env: { TOKEN: ${{ secrets.DEPLOY_TOKEN }} }`. Moved as is into a composite action, it would fail,

since the `secrets` context is unavailable there. A behaviour-preserving extraction must instead turn the secret into an input of the action and pass it explicitly from the caller:

```yaml
# .github/actions/deploy/action.yml
inputs:
  token: { required: true }
runs:
  using: composite
  steps:
    - run: ./deploy.sh
      shell: bash
      env: { TOKEN: "${{ inputs.token }}" }

# calling job
- uses: ./.github/actions/deploy
  with: { token: "${{ secrets.DEPLOY_TOKEN }}" }
```

Bibliography

[1] CARDOEN, G., MENS, T., AND DECAN, A. An empirical analysis of code clones in GitHub Actions workflows. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (2026), IEEE.

[2] FOWLER, M. *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.

[3] KHATAMI, A., WILLEKENS, C., AND ZAIDMAN, A. Catching smells in the act: A GitHub Actions workflow investigation. In *Proceedings of the 24th IEEE International Conference on Source Code Analysis and Manipulation (SCAM)* (2024), IEEE.

[4] ONSORI DELICHEH, H., CARDOEN, G., DECAN, A., AND MENS, T. Automation and reuse practices in GitHub Actions workflows: A practitioner's perspective. *ACM Transactions on Software Engineering and Methodology* (2026).

[5] ROSTAMI MAZRAE, P., DECAN, A., AND MENS, T. gawd: A differencing tool for GitHub Actions workflows. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR), Data and Tool Showcase Track* (2024), ACM.